Apiário Tech

Plataforma IoT para Monitorização e Gestão de Apiários

Documento de Apresentação

Responsável: Patrícia Silva

Versão: 1.0

Data: 22 de julho de 2026

Classificação: Confidencial — Uso interno

Repositório: github.com/patricia-sousa-silva/apiario-tech

Índice

1. Sumário Executivo
2. Contexto e Problema
3. Visão do Produto
4. Arquitetura do Sistema
5. Stack Tecnológica
6. Modelo de Dados
7. Funcionalidades Previstas (MVP)
8. Contrato de Telemetria
9. Ponto de Situação Atual
10. Trabalho em Falta
11. Fases do Projeto
12. Riscos e Mitigação
13. Segurança
14. Próximos Passos

1. Sumário Executivo

O **Apiário Tech** é uma plataforma IoT de monitorização apícola que combina sensores instalados no apiário, recolha local através de microcontrolador ESP32 e uma aplicação web para transformar leituras técnicas em informação compreensível e útil para a gestão de colmeias.

3 Componentes principais (Firmware, Backend, Web)	**8** Modelos de dados na base de dados
20+ Endpoints REST implementados	**6** Sketches de teste de firmware

O projeto encontra-se em **fase de desenvolvimento ativo**, com o backend e a estrutura base do frontend funcionais, e o firmware de teste preparado para validação em bancada.

2. Contexto e Problema

A avaliação do estado de uma colmeia depende habitualmente de visitas presenciais ao apiário e de observações realizadas durante as inspeções. Entre visitas, alterações de temperatura, humidade, peso ou funcionamento do equipamento podem passar despercebidas.

Problemas identificados

- Registos manuais dispersos e difíceis de analisar;
- Impossibilidade de detetar anomalias entre visitas;
- Ausência de histórico estruturado de leituras e intervenções;
- Não existe ligação permanente à Internet no apiário;
- Necessidade de autonomia energética e de armazenamento local.

Contexto do piloto

Parâmetro	Valor
Número de colmeias	2
Controlador	ESP32
Alimentação	Painel solar + bateria
Conectividade	Não permanente (importação CSV manual)

3. Visão do Produto

O Apiário Tech permitirá ao utilizador responder rapidamente a perguntas como:

- Qual é o estado atual de cada colmeia?
- Como evoluíram a temperatura, a humidade e o peso?
- Existem leituras anómalas que justificam atenção?
- Que inspeções, alimentações ou tratamentos foram realizados?
- O dispositivo está a recolher dados de forma regular?

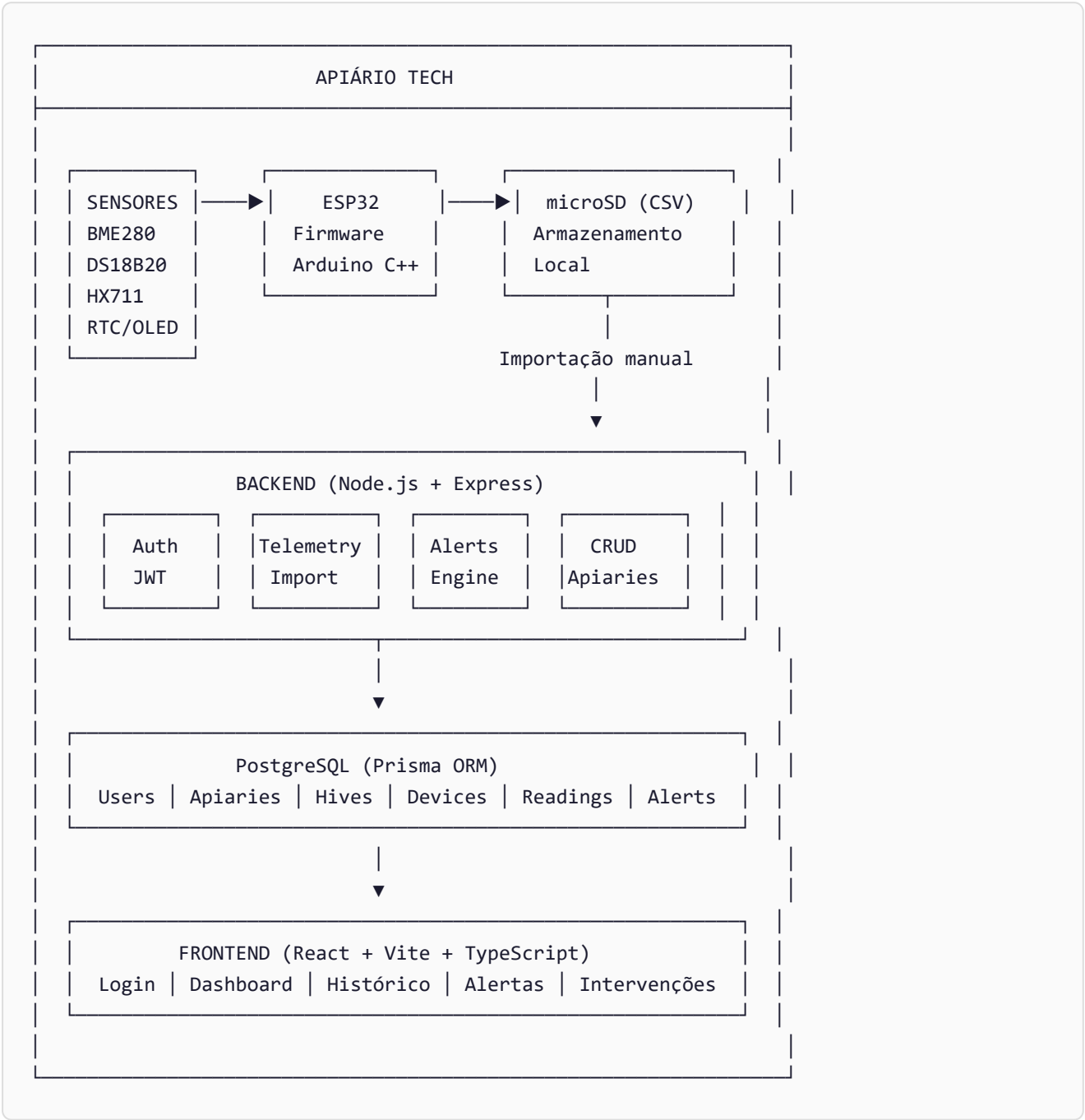
Objetivos principais

- Centralizar a informação relativa aos apiários e colmeias;
- Recolher e conservar telemetria com data e hora fiáveis;
- Apresentar o estado atual e o histórico de forma clara;
- Apoiar o acompanhamento através de alertas básicos;
- Manter registo cronológico das intervenções;
- Funcionar durante falhas de Internet e interrupções de energia;
- Permitir evolução gradual sem reconstruir a solução.

Objetivos de longo prazo

- Transmissão automática via 4G, LoRa ou Wi-Fi;
- Alertas avançados baseados em padrões históricos;
- Deteção e acompanhamento da Vespa velutina;
- Modelos de inteligência artificial validados.

4. Arquitetura do Sistema



Fluxo de dados

1. Sensores recolhem dados ambientais e do ninho;
2. ESP32 valida, formata e grava em CSV no microSD;
3. Utilizador importa o ficheiro CSV para a plataforma;
4. Backend valida, deduplica e persiste na base de dados;

5. Frontend apresenta dashboard, histórico e alertas.

5. Stack Tecnológica

Camada	Tecnologia	Justificação
Backend	Node.js + Express 5 + TypeScript	Ecossistema maduro, tipagem forte, async nativo
ORM	Prisma 7	Type-safe, migrations automáticas, adapter pattern
Base de dados	PostgreSQL 16	Fiabilidade, JSON nativo, full-text search
Autenticação	JWT + bcrypt	Stateless, seguro, padrão da indústria
Validação	Zod v4	Schema-first, integração com TypeScript
Frontend	React 19 + Vite + TypeScript	Componentização, HMR rápido, tipagem
Routing	React Router	SPA routing com proteção de rotas
HTTP Client	Axios	Interceptors para token automático
Gráficos	Recharts	Baseado em React, responsivo
Firmware	Arduino C++ (ESP32)	Compatível com bibliotecas de sensores
Upload CSV	Multer + csv-parse	Processamento em stream, memória eficiente

6. Modelo de Dados

O schema da base de dados inclui **8 modelos** interligados:

Modelo	Responsabilidade	Relações
User	Utilizadores e autenticação (email, password hash, role)	1:N → Apiary
Apiary	Apiários (nome, localização, coordenadas GPS)	N:1 → User; 1:N → Hive
Hive	Colmeias (nome, posição, estado ativo)	N:1 → Apiary; 1:N → Device, Reading, Alert, Intervention
Device	Dispositivos ESP32 (ID único, modelo, firmware)	N:1 → Hive; 1:N → Reading
TelemetryReading	Leituras de telemetria (10 métricas + validação)	N:1 → Device, Hive
AlertRule	Regras de alerta (métrica, operador, limiar)	1:N → Alert
Alert	Alertas gerados (mensagem, valor, reconhecimento)	N:1 → AlertRule, Hive
Intervention	Intervenções (inspeção, alimentação, tratamento, colheita)	N:1 → Hive

> **Regra de deduplicação:** A combinação `(device_id, timestamp)` é única. Importações repetidas do mesmo ficheiro não criam duplicados.

7. Funcionalidades Previstas (MVP)

Área	Funcionalidade	Estado
Acesso	Registo e autenticação de utilizadores (JWT)	✓ Implementado
Organização	Gestão de apiários (CRUD completo)	✓ Implementado
Organização	Gestão de colmeias (CRUD completo)	✓ Implementado
Dispositivos	Registo e gestão de dispositivos ESP32	✓ Implementado
Telemetria	Importação CSV com validação e deduplicação	✓ Implementado
Telemetria	Consulta de histórico com filtros e paginação	✓ Implementado
Telemetria	Última leitura válida por dispositivo	✓ Implementado
Alertas	Regras de alerta configuráveis (CRUD)	✓ Implementado
Alertas	Listagem e reconhecimento de alertas	✓ Implementado
Intervenções	Registo de inspeções, alimentações, tratamentos	✓ Implementado
Frontend	Página de Login e Registo	✓ Implementado
Frontend	Dashboard com última leitura	✓ Implementado
Frontend	Histórico com gráficos interativos	◐ Em desenvolvimento
Frontend	Página de importação CSV	◐ Em desenvolvimento
Frontend	Gestão visual de colmeias	◐ Em desenvolvimento
Frontend	Página de intervenções	◐ Em desenvolvimento
Firmware	Firmware base integrado (todos os sensores)	○ Pendente
Firmware	Testes de bancada documentados	○ Pendente
Comunicação	Envio automático via 4G/LoRa/Wi-Fi	○ Fase futura

8. Contrato de Telemetria (v1.0)

Formato único de telemetria utilizado tanto na exportação CSV pelo ESP32 como na futura receção automática via API.

Colunas do CSV

#	Coluna	Tipo	Unidade	Obrigatório
1	device_id	string	—	Sim
2	timestamp	datetime	ISO 8601 UTC	Sim
3	temp_ext_c	float	°C	Sim
4	humidity_pct	float	%	Sim
5	pressure_hpa	float	hPa	Sim
6	temp_nest1_c	float	°C	Sim
7	temp_nest2_c	float	°C	Sim
8	weight_g	integer	g	Não
9	battery_mv	integer	mV	Não
10	status_flags	string	—	Sim

Códigos de estado

Código	Significado
OK	Todos os sensores funcionaram normalmente
BME280_FAIL	BME280 não respondeu
BME280_STALE	Leitura não variou desde o ciclo anterior
DS18B20_1_FAIL	Sonda do ninho 1 não respondeu
DS18B20_2_FAIL	Sonda do ninho 2 não respondeu
RTC_FAIL	RTC não disponível
SD_FAIL	Erro de escrita no microSD

HX711_FAIL	Leitura do peso falhou
BATTERY_LOW	Tensão da bateria abaixo do limiar

> **Valor sentinela:** `-127.00` indica falha de sensor. O backend rejeita valores sentinelas como medições válidas e marca a leitura com o estado de falha correspondente.

9. Ponto de Situação Atual

Backend — Funcional

- API REST completa com 20+ endpoints;
- Autenticação JWT com registo e login;
- CRUD de apiários, colmeias, dispositivos e intervenções;
- Importação CSV com validação, deduplicação e relatório de erros;
- Motor de regras de alerta;
- Base de dados PostgreSQL com migrations e seed;
- Compilação TypeScript sem erros.

Frontend — Parcialmente funcional

- Estrutura React + Vite + TypeScript configurada;
- Sistema de autenticação com Context API e persistência;
- Páginas de Login e Registo funcionais;
- Dashboard com última leitura de telemetria;
- Layout com navegação e menu de utilizador;
- Páginas de histórico, importação e intervenções em desenvolvimento.

Firmware — Preparado para teste

- 6 sketches de teste isolados criados (BME280, DS18B20, OLED, RTC, microSD, HX711);
- Código original preservado e inventariado (3 versões);
- Contrato de telemetria v1.0 definido;
- Firmware base integrado ainda não desenvolvido (aguarda validação em bancada).

Documentação — Completa para a fase atual

- Visão e âmbito do produto;
- Estado atual do protótipo e inventário de hardware;
- Contrato de telemetria v1.0;
- Inventário do firmware original;
- Plano de validação em bancada (11 testes).

10. Trabalho em Falta

Frontend (curto prazo)

Tarefa	Prioridade	Complexidade
Página de histórico com gráficos Recharts	Alta	Média
Página de importação CSV com upload e feedback	Alta	Média
Gestão visual de colmeias e dispositivos	Média	Média
Página de intervenções com formulário	Média	Baixa
Página de alertas e regras	Média	Média
Design responsivo e melhorias de UX	Baixa	Média

Firmware (médio prazo)

Tarefa	Prioridade	Dependência
Validação em bancada dos 6 sensores	Crítica	Acesso ao hardware
Resolução do problema do microSD	Crítica	Teste isolado
Firmware base integrado	Alta	Sensores validados
Calibração da balança (HX711)	Alta	Montagem física
Teste contínuo de bancada (72h+)	Alta	Firmware integrado

Infraestrutura (médio/longo prazo)

Tarefa	Prioridade	Nota
Testes automatizados (unitários e integração)	Alta	Backend e frontend
Deploy em produção (cloud ou VPS)	Média	Após MVP estável
Comunicação remota (4G/LoRa/Wi-Fi)	Baixa	Fase futura
Aplicação móvel ou PWA	Baixa	Fase futura

11. Fases do Projeto

Fase	Entrega principal	Estado
0 — Consolidação	Inventário, código existente, documentação	✓ Concluída
1 — Firmware base	Sensores, OLED, RTC e microSD estáveis	○ Pendente (aguarda bancada)
2 — Peso	Balança calibrada e testada	○ Pendente
3 — Contrato de dados	Telemetria e CSV definidos	✓ Concluída
4 — Núcleo web	Utilizadores, apiários, colmeias, BD	✓ Concluída
5 — Percurso completo	ESP32 → CSV → importação → dashboard	◐ Parcial (backend pronto)
6 — Operação	Histórico, gráficos, alertas, intervenções	◐ Em desenvolvimento
7 — Validação	Segurança, testes, piloto prolongado	○ Pendente

12. Riscos e Mitigação

Risco	Impacto	Probabilidade	Mitigação
microSD instável	Perda de telemetria local	Alta	Teste isolado antes de integrar; cartão de qualidade industrial
Leituras congeladas (BME280)	Decisões com dados inválidos	Média	Deteção de stale data; validação temporal
Falha de energia	Corrupção de dados	Média	Escrita segura; teste de reinícios
Ausência de Internet	Atraso na consulta	Certa	Armazenamento local + importação CSV
Deriva da balança	Peso pouco fiável	Média	Calibração regular; testes com cargas conhecidas
Crescimento do âmbito	Atraso no MVP	Média	Funcionalidades futuras fora do MVP
Acesso indevido	Exposição de dados	Baixa	JWT, bcrypt, validação Zod, HTTPS

13. Segurança

- **Autenticação:** JWT com expiração configurável;
- **Palavras-passe:** Hash com bcrypt (10 rounds);
- **Validação:** Todos os inputs validados com Zod antes de processar;
- **Segredos:** Nunca no código-fonte; variáveis de ambiente (.env);
- **CORS:** Configurado para origens autorizadas;
- **Dados:** Leituras não são eliminadas sem rastreabilidade;
- **Autorização:** Cada utilizador acede apenas aos seus apiários.

14. Próximos Passos

Imediato (1-2 semanas)

1. Completar páginas do frontend (histórico com gráficos, importação CSV);
2. Push do código para o GitHub;
3. Testes de integração básicos.

Curto prazo (2-4 semanas)

1. Validação em bancada dos sensores (junto ao ESP32);
2. Resolução do problema do microSD;
3. Desenvolvimento do firmware base integrado;
4. Primeiro percurso completo: ESP32 → CSV → Backend → Dashboard.

Médio prazo (1-3 meses)

1. Calibração da balança;
2. Teste contínuo de bancada (72h+);
3. Testes automatizados;
4. Deploy em ambiente de produção;
5. Piloto prolongado nas 2 colmeias.
